Contents

Cloud Platforms — Interview Questions & Answers **1**

Cloud Platforms — Interview Questions & Answers

Q1: Explain the difference between an IAM Role and an IAM User. When should you use each? > **IAM User**: A permanent identity with long-lived credentials (username/password or access keys). For humans that can't use SSO, or legacy systems that require static keys. Avoid wherever possible — keys must be rotated, can be leaked, hard to audit at scale. > > **IAM Role**: A temporary identity assumed by humans, AWS services, or external systems. Credentials are short-lived (15 min to 12 hours), automatically rotated by STS, and scoped to the session. Always use roles for EC2 instances, Lambda, ECS tasks, EKS pods (via IRSA), CI/CD pipelines (via OIDC). > > **Rule of thumb**: Every non-human identity should use a role. Every human should use SSO/Identity Center federating to roles — never IAM users with long-lived keys.

Q2: What is the difference between a Security Group and a Network ACL in AWS? > **Security Groups**: > - Stateful — if inbound traffic is allowed, return traffic is automatically allowed > - Attached to ENIs (network interfaces), not subnets > - Allow rules only (no explicit deny possible) > - Evaluated as a single set (all rules checked, most permissive wins) > - Source/destination can be another Security Group (great for service-to-service rules) > > **Network ACLs**: > - Stateless — must explicitly allow inbound AND outbound for the same connection > - Attached to subnets — affects all resources in the subnet > - Allow and Deny rules — can block a specific IP (Security Groups cannot) > - Rules evaluated in number order, first match wins > > **When to use NACLs**: Additional perimeter defense, blocking specific IPs/CIDRs, compliance requirements for subnet-level controls. Most teams use only Security Groups for day-to-day; NACLs as a "defense in depth" second layer.

Q3: How does AWS VPC peering work, and what are its limitations? > VPC Peering creates a private network route between two VPCs (same or different accounts/regions). Traffic stays on the AWS backbone — no internet gateway needed. > > **Limitations:** > - Non-transitive: if VPC A peers with B and B peers with C, A cannot reach C through B. Must create A-C peer separately. > - No overlapping CIDR blocks — peer VPCs cannot have overlapping IP ranges (plan CIDRs centrally) > - No edge-to-edge routing (can't use a VPC peer as a gateway to the internet or VPN) > > **Alternatives for hub-and-spoke at scale:** > - AWS Transit Gateway: connects hundreds of VPCs and on-prem networks, supports transitive routing, centralized routing tables, bandwidth up to 50 Gbps per attachment > - AWS PrivateLink: for exposing specific services privately without full VPC peering

---

Q4: Explain S3 consistency model. What changed in 2020? > Before December 2020: S3 provided *eventual consistency* for overwrite PUTs and DELETEs. If you wrote to an object and immediately read it, you might get the old version. > > Since December 2020: S3 provides **strong read-after-write consistency** for all operations — PUTs, DELETEs, LIST. A write followed immediately by a read returns the new data. LIST operations after a PUT show the new object. > > This eliminated many workarounds that applications had to implement. Relevant for: state file storage, data pipelines, distributed coordination patterns built on S3.

Q5: What is an EC2 Instance Profile and how does it differ from directly assigning an IAM role? > An **Instance Profile** is a container for a single IAM role that can be attached to an EC2 instance. The terms are often used interchangeably, but they are technically distinct: > - IAM Role: the identity with policies attached > - Instance Profile: the mechanism that associates a role with an EC2 instance > > When you create a role for EC2 in the console, AWS automatically creates an Instance Profile with the same name. Via CLI/Terraform, you must create them separately. > > The Instance Metadata Service (IMDS) at `169.254.169.254` serves the temporary credentials to code running on the instance. Applications using the AWS SDK automatically call this endpoint — no credentials configuration needed in the application.

Q6: Compare RDS Multi-AZ vs Read Replicas. When do you need each? > **Multi-AZ (High Availability):** > - Synchronous replication to a standby in a different AZ > - Standby is NOT accessible for reads — purely

for failover > - Automatic failover in 60-120 seconds if primary fails (Aurora: < 30s) > - Use for: production databases where downtime is unacceptable > > **Read Replicas (Performance/Scale):** > - Asynchronous replication — slight lag (seconds to minutes) > - Replicas are readable — scale out read traffic > - Can be promoted to primary manually (not automatic) > - Up to 15 read replicas for Aurora, 5 for RDS > - Use for: reporting, analytics, read-heavy workloads > > **Use both**: Multi-AZ for HA, read replicas for scaling reads. Aurora Global Database gives you read replicas across regions with < 1 second replication lag.

Q7: What is AWS Savings Plans and how does it differ from Reserved Instances? > **Reserved Instances (RI):** > - Commit to a specific instance type, size, region for 1 or 3 years > - Up to 72% savings > - Zonal RI also reserves capacity (important for large instances) > - Scope is specific: `m5.xlarge` in `us-east-1a` — changes require Marketplace sale > > **Savings Plans:** > - Commit to a dollar amount of spend per hour for 1 or 3 years > - **Compute Savings Plans**: apply to any EC2 family, size, region, OS — even Lambda and Fargate. Most flexible. > - **EC2 Savings Plans**: specific instance family in a region — deeper discount (~66% max) vs Compute Plans (~66%) > - Cannot sell on Marketplace but automatically apply to matching usage > > **When to use**: Savings Plans are simpler and more flexible. Use Compute Savings Plans for mixed fleets. Only use RIs if you need capacity reservation guarantees.

Q8: Explain the AWS Shared Responsibility Model in the context of EKS. > AWS manages: the Kubernetes control plane (API server, etcd, scheduler), the underlying EC2 infrastructure for control plane nodes, OS patching of control plane, availability of the K8s API. > > You manage: > - EKS worker node OS updates and AMI patches (use managed node groups with auto-upgrade) > - Kubernetes add-on versions (CoreDNS, kube-proxy, VPC CNI) > - Pod security (SecurityContext, PSA, network policies) > - IAM/RBAC configuration for cluster access > - Container images and their vulnerability patches > - Application-level security, secrets management, data encryption > > With Fargate for EKS: AWS also manages the worker node infrastructure — you only manage pods and their configs.

Q9: How does CloudFront caching work? How do you invalidate cache? > CloudFront caches content at **edge locations** close to users. When a request arrives: > 1. Edge checks cache: cache **hit** → serve from edge (fast, no origin cost) > 2. Cache **miss** → forward to origin (S3, ALB, etc.), cache the response, serve to user > > **Cache control is determined by:** > - `Cache-Control: max-age=3600` header from origin > - CloudFront cache behavior settings (min/max/default TTL) > - `Vary` header (cache per Accept-Encoding, Accept-Language) > > **Invalidation:** > - `aws cloudfront create-invalidation --distribution-id ID --paths "/index.html" "/static/*"` > - Wildcards supported: `/*` invalidates everything (costs $0.005 per 1000 paths beyond first 1000/month free) > - Better strategy: use versioned filenames (`app.v2.js`) — old file naturally expires, no invalidation needed > > **Cache key customization**: Include/exclude query strings, headers, cookies from cache key to control granularity.

Q10: Your EKS pods are failing to pull images from ECR with an auth error at 2am. What's happening and how do you fix it? > **Most likely cause**: ECR credentials expired. The `ecr get-login-password` token is valid for **12 hours**. If you're using a static credential (wrong approach), it expires. > > **Correct solution**: Use the **Amazon ECR credential helper** or the **VPC ECR endpoint**: > - EKS nodes have an instance profile with `ecr:GetAuthorizationToken` and `ecr:BatchGetImage` permissions > - The `credential-provider` plugin in the kubelet config handles token refresh automatically > - Verify with: `aws ecr get-authorization-token --region us-east-1` > > **Diagnose:** > 1. `kubectl describe pod FAILING_POD` — look at Events section for `ErrImagePull` / `ImagePullBackOff` > 2. Check node IAM role has ECR read permissions > 3. Check if ECR is in the same region as the cluster > 4. Verify VPC endpoint for ECR exists (if cluster is in private VPC with no internet) > 5. For cross-account ECR: check ECR repository policy allows the source account's node role